

Case Study: The origin of human malaria

Dag Ahren, Malou Arvidsson & Björn Canbäck

Report

Please send in the following through Canvas:

- A file with a README documenting your work in a reproducible way. Make sure that you describe your choice of filters AND describe your reasoning for those choices. Each command should be present and links to downloaded data (including versions of data and software).
- A short report answering the questions from the exercise.

We will discuss the results on Friday.

Data Collection

Background

Malaria is caused by parasites belonging to the phylum *Apicomplexa*. One of the more well known genera causing malaria is *Plasmodium*. The species of this genus infect not only humans, but also other mammals, birds and “reptiles”. The single one species that causes most malaria infections in humans is *Plasmodium falciparum*. This is also the most serious of malaria infections. Many genomes of *Plasmodium* have been sequenced. Recently the first genome of an avian (bird) malaria parasite has been sequenced. The species’ name is *Haemoproteus tartakovskyi*. The genomes we will work with are (empty cells to be filled later):

#	Species	Host	Genome size	Genes	Genomic GC
1	<i>Plasmodium berghei</i>	rodents			
2	<i>Plasmodium cynomolgi</i>	macaques			
3	<i>Plasmodium falciparum</i>	humans			
4	<i>Plasmodium knowlesi</i>	lemures			
5	<i>Plasmodium vivax</i>	humans			
6	<i>Plasmodium yoelii</i>	rodents			
7	<i>Haemoproteus tartakovskyi</i>	birds			
8	<i>Toxoplasma gondii</i>	humans			

There is a controversy among researches whether *Plasmodium falciparum* is

related to the other mammalian parasites, or if it originates from a bird malaria parasite which has changed its host. We may solve this by making an extensive phylogenetic analysis.

1. **Do you think that in a phylogenetic tree the parasites that use similar hosts will group together?**

***Plasmodium* data**

The first task will be performed on the server. Remember to organize your files! All the genomes of the *Plasmodium* species together with the outgroup taxon *Toxoplasma gondii*, another member of **Apicomplexa**, are found at the course web site in one single file. The file name is **plasmodiumGenomes.tgz**. With one of the genome files you will later encounter a problem that you have to solve.

First a gene prediction using **GeneMark** should be carried out (the program name is **gmes_petap.pl**, see the gene prediction and annotation exercise from the previous course). Since prediction takes time, divide the work among the students so that one particular student makes the gene prediction for one particular genome. The genome of *Toxoplasma gondii* takes too long to run and can be skipped. The produced gff-files should be copied to the **/tmp/Prediction** directory on the course server. From here, all students can copy the gff-files. The *Toxoplasma* gff-file can be downloaded from the course web site with the name **Tg.gff**.

Processing of *Haemoproteus tartakovskyi* data

Clean genome sequence

The *H. tartakovskyi* genome was sequenced by the 454 technology. Both shotgun and paired end sequencing were used. The scaffold file is found on the course server and has the name **Haemoproteus_tartakovskyi.genome**. Since the input reads derive both from the bird and the parasite, bird scaffolds should be removed. Select an appropriate GC-content (see presentation) and remove scaffolds above this threshold. Also remove scaffolds that are less than 3000 nucleotides. Try to make the script yourself, if not, you can download a python-script from the course server at **/resources/binp29/Data/malaria/removeScaffold.py**. You will do an additional filtering after gene prediction.

Make a gene prediction

With the new genome file, make a gene prediction. You will probably still have some scaffolds that derive from the bird. These should be short.

2. **Why?**

To create **fasta** sequences from the **gff** file and the genome file use **gffParse.pl** found at the course server. It's recommended that you use the **-c** option.

This didn't work!

Compare the first column in the **gff**-file with the headers in the genome fasta file. Do you see what is wrong? Try to adjust it, otherwise the solution is here:

```
cat Haemoproteus.gff | sed "s/ GC=.*\tGeneMark.hmm/\tGeneMark.hmm/" > Ht2.gff
# Haemoproteus.gff should be changed to the name you use.
```

Which of the two programs, in your opinion, is to blame? Remove scaffolds that have genes that are from avian origin. For this purpose use **blastx** or **blastp** on the course server. Use **SwissProt** as database. It is already installed with that name (**SwissProt**). If you want to look for the path to the BLAST databases, do:

```
echo $BLASTDB
```

If this environmental variable is set you don't have to specify the path, it's enough giving the database name like: **-db SwissProt**. As you experience the BLAST search takes a lot of time. SwissProt's size is only 1% to that of UniProt so even on a more powerful computer the search will take considerable time. If you don't want to wait download the **Ht.blastp** file from the course server. Since the students have chosen different thresholds for the GC-content, the BLAST output file maybe not reflects the fasta file entirely.

Use the taxonomy for the top hit in the BLAST output to decide if the query sequence originates from the host. In SwissProt, the five characters after the underscore is a species abbreviation (what is before the underscore?).

Optional, but very good from a practical point of view: make a script for this by using the information from the NCBI taxonomy file which you download like (don't download to the server):

```
wget ftp://ftp.ebi.ac.uk/pub/databases/taxonomy/taxonomy.dat
```

and the SwissProt dat file:

```
wget ftp://ftp.uniprot.org/pub/databases/uniprot/
current_release/knowledgebase/complete/uniprot_sprot.dat.gz
```

The files are also provided in **/resources/binp29/Data/malaria/**. The files are approximately 4GB, so instead of copying these files you can create a symlink. It is done like:

```
ln -s /resources/binp29/Data/malaria/taxonomy.dat taxonomy.dat
```

This will create a link to the file from the current directory. do the same with **uniprot_sprot.dat**. Check that it worked using **ls -l**

A script is provided to retrieve the host scaffolds if you choose not to try it yourself (you should though, it is fun!). The script is **datParser.py** and is found

on the server at `/resources/binp29/Data/malaria/`.

The script is run like (example file names):

```
python datParser.py Ht.blastx Ht.fna taxonomy.dat uniprot_sprot.dat > scaffolds.txt
```

The program prints to standard output, so redirect to a file if you want to store it.

Write a python script, or use **bash** to clean the fasta file from bird scaffolds.

After removal of scaffolds containing host genes, you probably want to do a new gene prediction so you don't have to edit the current one.

3. **Insert the missing data in the above table. Use bash, not internet!**
4. **Compare the genome sizes with other eukaryotes and bacteria. Discuss with your partner (that is student partner) the reason for the observed genome sizes.**
5. **What may cause the biased GC-contents in some of the species?**

Phylogenetic trees

Identify orthologs

Species trees are in the majority of cases inferred from gene or protein trees. However, it is today well known that a single gene may show a different topology than the "true" species tree. One important criteria when using genes for reconstructing a species tree is that orthologous genes, or in short orthologs, are used. Orthologs are descendants from the same gene in an ancestor of the two species. Paralogous genes, or paralogs, derive from a gene duplication *within* the species. It is complicated to identify orthologs. There are four main tools used for this purpose:

- OrthoMCL (probably the most well known)
- InParanoid
- proteinortho
- BUSCO

We will use the last two. **proteinortho** uses a reciprocal BLAST or Diamond search to find orthologs, while **BUSCO** uses **HMM** to identify a core set of proteins that normally should be found in the lineage that the species under investigation belong to. In the case of **BUSCO** input sequences could be a genome sequence, a transcriptome or a proteome. We will compare the output of the two programs. Print the protein sequences in fasta format for each genome with the aid of the **gffParse.pl** program downloaded earlier. Use the **-b** option to give the protein file a shorter name as well as the **-c** option. Use **Pb** for *Plasmodium berghei*, **Pc** for *Plasmodium chabaudi* and so on. Then install **proteinortho**. This can be

done with conda. More details about the program and installation can be found at: <https://www.bioinf.uni-leipzig.de/Software/proteinortho/>

The software may be run as:

```
nohup proteinortho6.pl {Ht,Pb,Pc,Pf,Pk,Pv,Py,Tg}.faa &
```

6. What does the curly braces notation stand for?

While running take a look at the output files, especially `myproject.blast-graph`. If you run `top` and then type `c` you will see the entire `blast` command that `proteinortho` uses. (*note*: On the new server, the `proteinortho` run will be fast if many cores are free, so you might not ‘get time’ to do this inspection)

When the first `blast` results have been produced inform the teacher and continue with the BUSCO analysis. Busco can be installed using conda. More information about the software and installation can be found at <https://busco.ezlab.org/>.

Example on how to run BUSCO:

```
busco -i ../Fasta/Pb.faa -o Pb -m prot -l apicomplexa
```

what does the flag `-l` stands for? and why do we choose *apicomplexa*?

This time also *Toxoplasma* should be run. Take a look at the `full_table_??.tsv` files. We will only use the proteins that have been flagged as **Complete** or **Duplicated**. When it comes to the duplicated genes chose only one of these so that you in the end get one to one orthologs.

7. Compare how many BUSCOs (orthologues proteins) that are found in each proteome. Do the investigated parasites have close to complete numbers of BUSCOs?
8. Do you think that the assembly of the *Haemoproteus tartakowskyi* genome is a reasonable approximation of the true genome?
9. How many of the BUSCOs are found in all eight organisms?
10. If *Toxoplasma* is removed, how many BUSCOs are shared among the remaining seven species. Interpret!

Apparently, we can’t use the *Toxoplasma* data. We included this species to serve as an outgroup for the phylogenetic analysis. By using `bash` or `Python`, create one protein fasta file per BUSCO based on the eight `full_table_??.tsv` files. In each file all eight species should be present with one sequence. We are only using one to one orthologs (why?). For information on what gene corresponds to a particular BUSCO, look in the directory `hmmmer_output`. Use the same sequence id for the same organism in all created files, like `>Pf`. Output file names should probably be the names of the BUSCOs.

Alignments and trees

We will now make alignments for all individual BUSCO fasta files. Use `clustalo` for this. Then run `raxml` for all the alignments. Make a shell script for this. The trees can be visualized at: <http://itol.embl.de/>

You can install `clustalo` and `BUSCO` using `conda`:

```
conda install -c bioconda clustalo raxml
```

Run `clustalo` on the BUSCO protein fasta files you created. example:

```
clustalo -i EPrGT05000638.faa -o EPrGT05000638_aligned.faa -v
```

An example run for `raxml` could be:

```
raxmlHPC -s EPrGT05000638_aligned.faa -n EPrGT05000638.tre -o \
Tg -m PROTGAMMABLOSUM62 -p 12345
```

The `-o` is the option for specifying an outgroup. Here we choose *Toxoplasma*. Why? If you excluded *Toxoplasma* from the alignment files, choose another suitable outgroup.

Use `consense` from the `phylip` package to merge all individual trees

```
conda install -c bioconda phylip
```

11. Does all protein trees reflect the “true” species tree?
12. What is the phylogenetic position of *Plasmodium falciparum*?
13. Do you think that the GC contents have an impact on the tree topology?
14. Do you think that the host range has an impact on the tree topology?
15. Are the BUSCO proteins also found as orthologs in the `proteinortho` output?
16. Make a script that concatenates the alignments for each organism and BUSCO into one fasta file that in the end should contain seven sequences. Alternatively, use `bash`.
17. Make a tree of this “superalignment”. Does it correspond to the `consense` tree?

Algorithm development

The result in the last exercise is likely biased due to e.g. signal saturation, convergent evolution or bias resulting from base composition. We need to verify the results. One way of doing this is to investigate indels in alignments. We will use the absence or presence of an indel as an informative character state and use these states (indels) to build a phylogenetic tree. If two or more lineages (species) share indels, this could be an indication of relatedness.

We have created global alignments and created phylogenetic trees based on those alignments. However in a global alignment there will be segments of nucleotides or amino acids that are poorly aligned. Poorly aligned positions should not be used in phylogenetic analysis because we can't be sure that they share a common origin.

You can view an alignment using the software **seaview**. You can either install it on the server and log in using **ssh -X**, or download the BUSCO alignment files to your local computer and use it there. **seaview** is found here: <http://doua.prabi.fr/software/seaview>.

One popular software for removing “bad” alignment segments is **Gblock**. Gblock is part of the seaview package and it works from anywhere if you exported your seaview folder to your PATH variable.

GBlocks by default removes all positions that contain gaps. Since we will use indels as informative markers we have to use less stringent settings, like:

```
Gblocks BUSCO.alignment -t=p -b5=all
```

18. Run Gblocks on your BUSCO alignments.

Your task is to write a program that outputs binary strings representing the absence/presence of indels:

```
>Pf
00101100
>Pc
00010010
>Pb
11011000
>Pk
11011011
```

Positions with only one character state should be excluded. The program should take any number of (alignment) fasta files as input, process them one by one, and finally merge the output into a single binary string for each organism. This string could be processed with:

```
raxmlHPC -m BINGAMMA -p 12345 -s binaryAlignment.fasta -n binary.tre
```

The size of the indel should not matter. A gap spanning several positions will still be considered as one gap. Positions in the beginning or the end containing gaps should be removed. An example (amino acids/nucleotides are replaced with X:s):

```
0123456789 # This is not part of the input
>Pf
X-XXXXXX-
>Pc
X-XXXXXX-
```

```
>Pb
XXX-X--XXX
>Pk
XXX-XX--XX
```

First remove end positions with gaps. Then position 0-8 will remain. In Pb and Pk the gaps in the second indel don't overlap and this indel should be excluded. The result will be (if a gap is represented by 0):

```
>Pf
01
>Pc
01
>Pb
10
>Pk
10
```

19. Compare the indel tree with the two in the earlier exercises.
20. Compare your results with the published study found at the course web server, directory **Articles**.